

 BLOCK 1 – DSA: Iterators & Generators (Lazy Evaluation)

 Core Concept

List → loads everything at once 

Generator → gives values one-by-one 

 Example

 normal list

```
def get_all_tokens():
```

```
    return ["Hello", " world", "!"]
```

 generator (stream)

```
def stream_tokens():
```

```
    yield "Hello"
```

```
    yield " world"
```

```
    yield "!"
```

 Usage

```
for chunk in stream_tokens():
```

```
    print(chunk, end="", flush=True)
```

 Output appears word-by-word 

 LeetCode Problem  #284 – Peeking Iterator

 Why this matters

LLM response = long text

Streaming → send token-by-token ✓

Memory efficient ✓

Better user experience ✓

🔍 Think

Normal chat → wait 5s → full answer ✗

Streaming → see words instantly ✓

Which feels faster? → Streaming ✓

🤖 BLOCK 2 – AI Concept: Streaming in FastAPI

✓ Idea

Instead of: LLM → full reply → return

We do: LLM → token → send → token → send → ...

✓ FastAPI tool

from fastapi.responses import StreamingResponse

✓ Simple streaming generator (NO GROQ / OLLAMA needed)

We simulate streaming:

```
import time
```

```
def fake_stream(text):
```

```
    for word in text.split():
```

```
        yield word + " "
```

```
        time.sleep(0.2)
```

👉 This lets you learn streaming concept without API

✅ Add this to your main.py

✅ Streaming endpoint

```
from fastapi.responses import StreamingResponse

import time

@app.post("/chat/stream")
async def stream_chat(req: ChatRequest):

    def generate():

        # simulate your actual response

        full_reply = f"Streaming response for: {req.message}"

        collected = ""

        for word in full_reply.split():

            token = word + " "

            collected += token

            yield token

            time.sleep(0.2)

        # after streaming, store in memory

        bot = get_session(req.user_id)

        bot.chat_history.append({"role": "assistant", "content": collected})

    return StreamingResponse(generate(), media_type="text/plain")
```

✅ ✅ STEP 4 — TEST STREAMING

✅ Run server

```
python -m uvicorn main:app --reload
```

✅ Test using browser or curl

✅ CURL (important)

curl -N -X POST <http://127.0.0.1:8000/chat/stream>

-H "Content-Type: application/json"

-d '{"user_id":"user1","message":"Explain BST"}'

👉 You will see:

Streaming response for: Explain BST

✅ appearing word-by-word

✅ ✅ VERIFY

Make sure:

✅ Words appear gradually (not all at once)

✅ /chat still works

✅ No crash

🔄 BLOCK 4 – Reflect

1. Why streaming feels faster?

Because user sees progress instantly ✅

2. Why ReAct is hard to stream?

ReAct needs full reasoning loop ❌

Streaming only outputs final tokens ✅

3. What if user disconnects?

Full reply may not be saved ❌

4. Which is better for demo?

/chat → simple ✅

/chat/stream → impressive ✅ 🔥

🚀 ✅ FINAL STATUS

After Day 1:

✓ Learned generators ✅

✓ Built streaming endpoint ✅

✓ Improved UX ✅

✓ No dependency on Groq ✅
